

Taller de Programación 2026



Prof: Víctor Bórquez H.
Esc: Ingeniería, Energía y
Tecnología

Temario

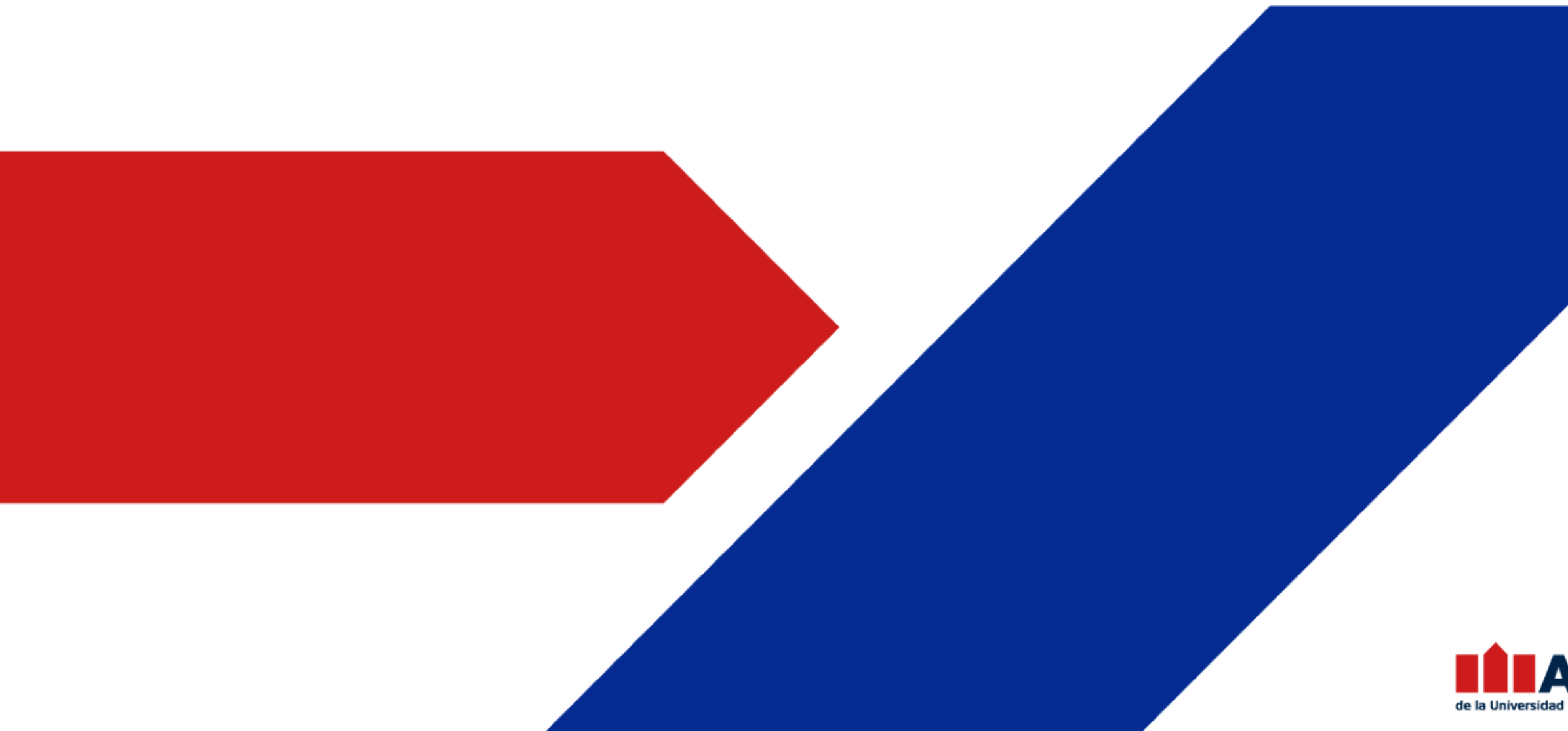
- Unidad 1: Introducción a la Programación Orientada a Objeto.
- Unidad 2: Diseño y Programación orientada a objeto.
- Unidad 3: Programación de interfaces gráficas.

Herramientas

- Visual Studio Code (VS Code)
- Python
- Star UML

Evaluaciones

- Certamen 1: 31/08/2026
- Certamen 2: 23/09/2026
- Certamen 3: 16/10/2026
- Certámenes de desarrollo en laboratorio.
- Tareas dan puntaje para las pruebas.

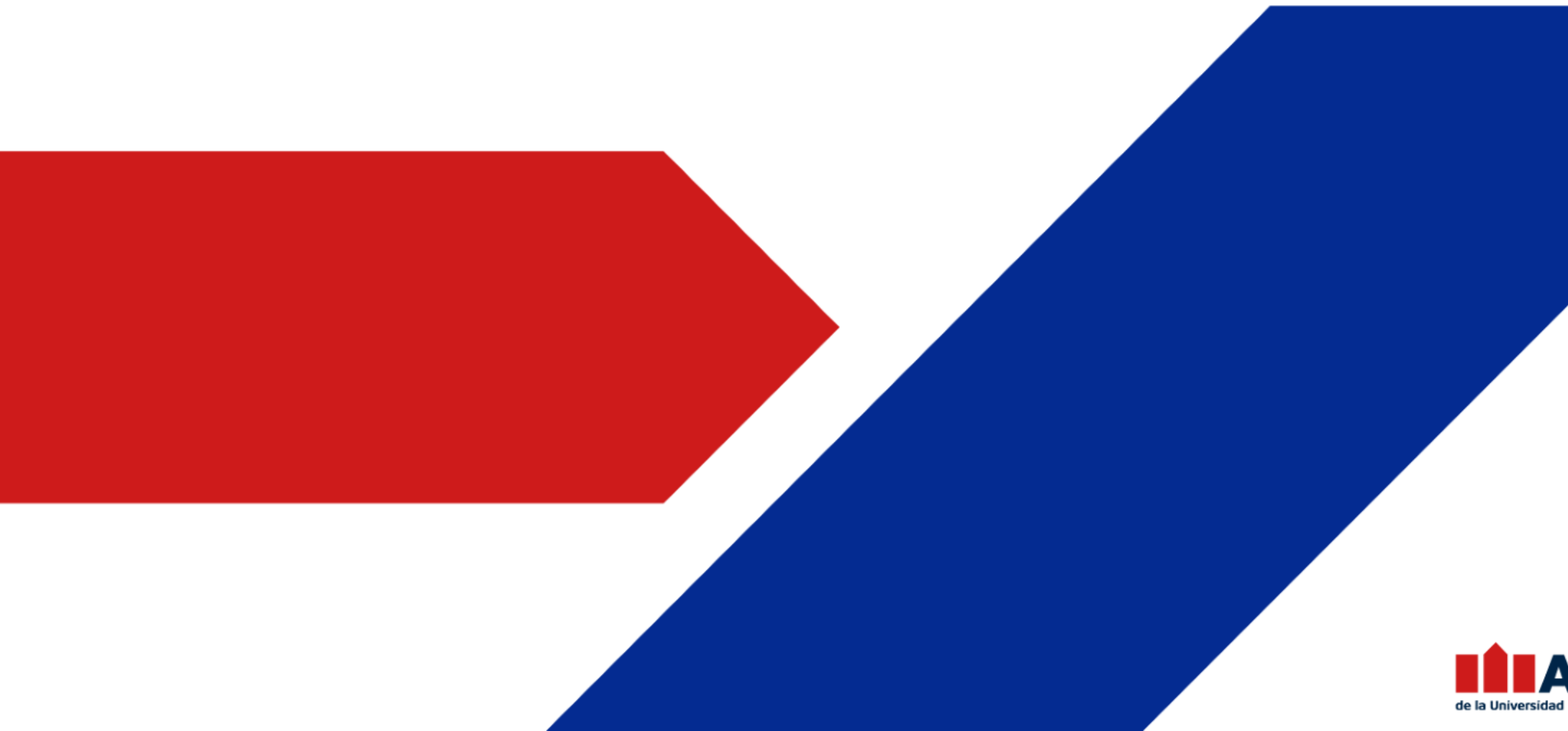


Unidad 1: Introducción a la Programación Orientada a Objetos

- Objetivo de aprendizaje
 - Al finalizar esta unidad, el estudiante será capaz de comprender el paradigma de Programación Orientada a Objetos (POO), identificar sus componentes principales y aplicar los conceptos fundamentales mediante ejemplos prácticos en Python.

Termario : Unidad 1 - Introducción a la Programación Orientada a Objetos

- 1. Introducción
- 2. Evolución de la Programación
- 3. Componentes de la Programación Orientada a Objetos
- 4. Propiedades de la Programación Orientada a Objetos
- 5. Ejemplo Integrador
- 6. Buenas Prácticas en Programación Orientada a Objetos
- 7. Actividades Prácticas
- 8. Resumen de la Unidad
- 9. Evaluación Formativa



Introducción

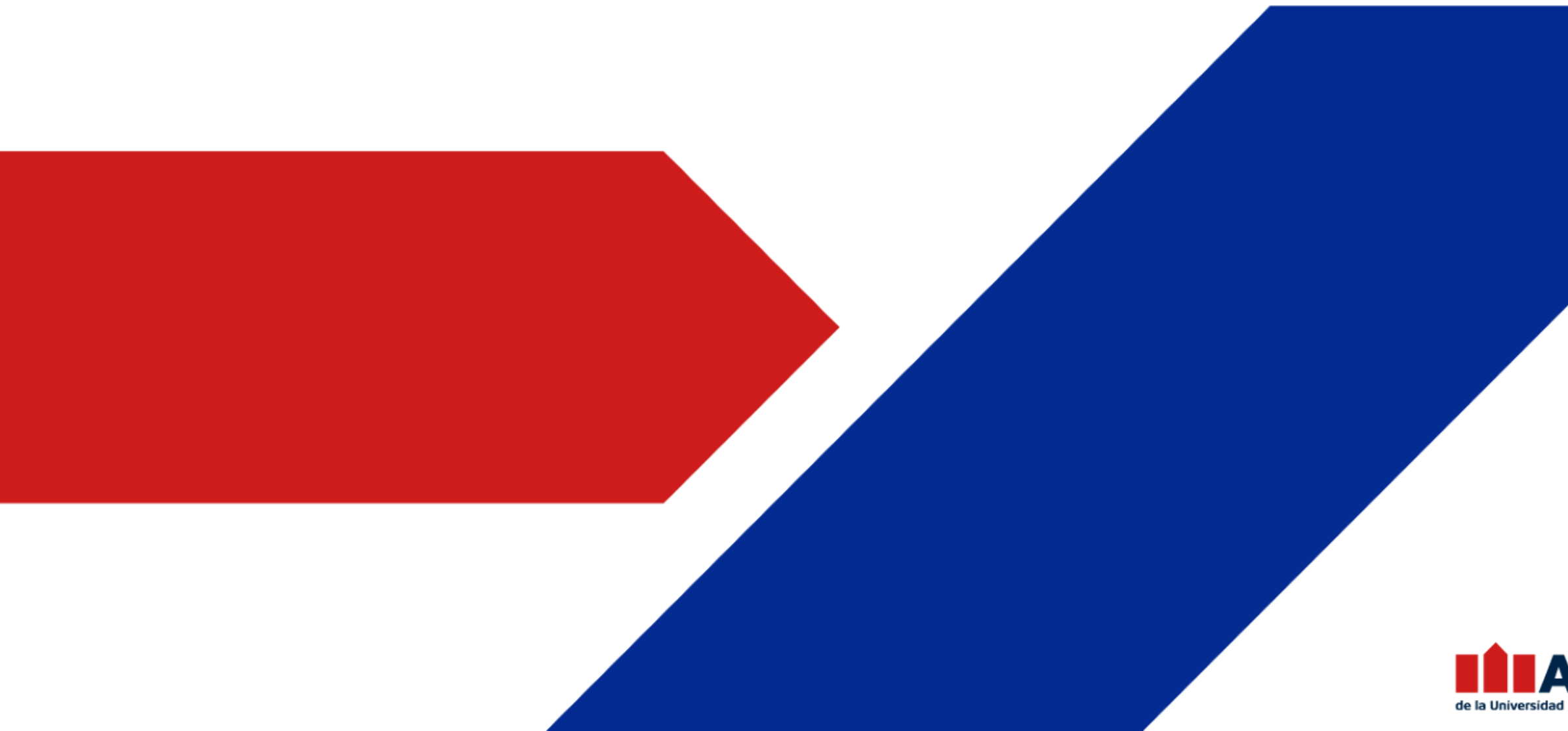
- Antes de aprender Programación Orientada a Objetos debemos entender por qué nació.
- Cuando comenzaron a desarrollarse los primeros programas (años 50-60), eran pequeños y resolvían tareas simples.
- Con el tiempo aparecieron sistemas enormes:
 - Sistemas bancarios
 - Sistemas hospitalarios
 - Videojuegos
 - Redes sociales
 - Sistemas de control industrial
- Los programas crecieron tanto que era difícil mantenerlos.
- La solución fue cambiar la forma de pensar la programación.

Introducción

- En lugar de pensar en funciones...
 - **"¿Qué hace el programa?"**
- comenzamos a pensar en objetos...
 - **"¿Quién hace el trabajo?"**
- Ese cambio revolucionó el desarrollo de software.

¿Qué es un paradigma de programación?

- **Un paradigma es una forma de resolver problemas.**
- Existen varios paradigmas:
 - Programación Lineal
 - Programación Estructurada
 - Programación Orientada a Objetos
 - Programación Funcional
- Nos enfocaremos en la Programación Orientada a Objetos.



Evolución de la programación

- **1. Programación Lineal**

- Fue el primer estilo de programación.
- Todo ocurría desde arriba hacia abajo.
- Ejemplo:
 - `nombre = input("Nombre: ")`
 - `edad = int(input("Edad: "))`
 - `print(nombre)`
 - `print(edad)`

Evolución de la programación

- Todo está mezclado.
- No existen módulos.
- No existen funciones.
- No existen objetos.
- Si el programa crece...
 - Se vuelve inmanejable.
- **Problema**
 - Imaginemos un sistema con:
 - 5000 líneas
 - 100 variables
 - 30 programadores
 - Encontrar un error sería muy difícil.

Evolución de la programación

- **2. Programación Estructurada**

- Aparecen:
 - Funciones
 - Procedimientos
 - Modularización
- Ejemplo:
 - `def calcular_area(base, altura):`
 - `return base * altura / 2`
 - `area = calcular_area(10,5)`
 - `print(area)`
- Ahora el código está organizado.
- Pero aún existe un problema...
- Los datos están separados de las funciones.

Evolución de la programación

- Ejemplo
 - nombre = "Juan"
 - edad = 25
 - def mostrar():
 - print(nombre)
 - mostrar()
- La función depende de variables externas.
- Eso provoca problemas cuando el sistema crece.

Evolución de la programación

- **3. Programación Orientada a Objetos**

- Ahora los datos y las funciones viven juntos.

- Ejemplo:

- class Persona:

- def __init__(self,nombre,edad):

- self.nombre = nombre

- self.edad = edad

- def mostrar(self):

- print(self.nombre,self.edad)

- La persona posee:

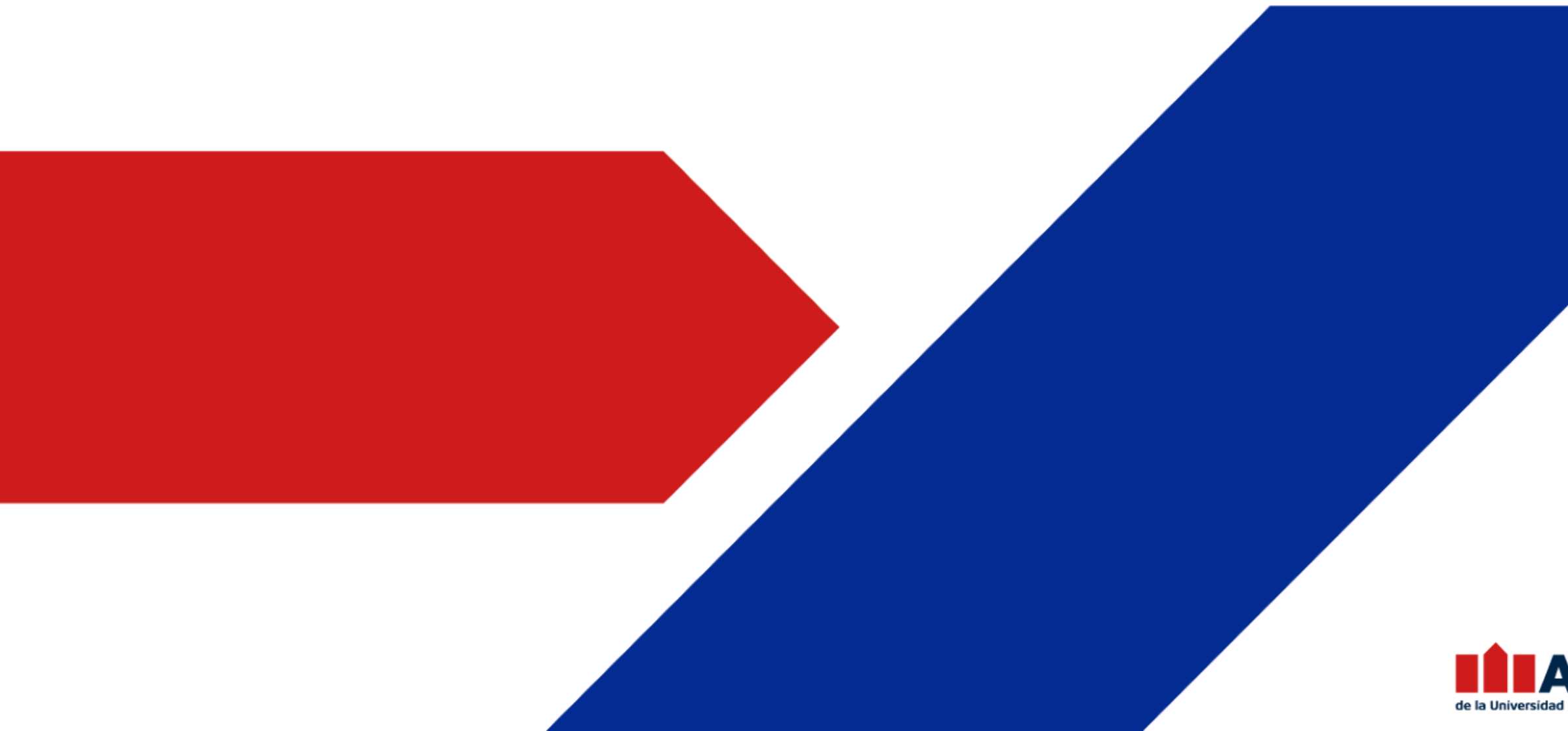
- Datos: nombre y edad

- Y además comportamientos: mostrar()

- Todo está dentro del mismo objeto.

Evolución de la programación

- Historia de la Programación Orientada a Objetos
- Década de 1960
- Lenguaje Simula 67
- Fue el primer lenguaje orientado a objetos.
- Introdujo:
 - Clases
 - Objetos
- Posteriormente aparecieron
 - Smalltalk
 - C++
 - Java
 - C#
 - Python
- Actualmente casi todos los lenguajes modernos soportan POO.



Componentes de la POO

- **¿Qué es un Objeto?**
- Un objeto representa una entidad del mundo real.
- Puede ser:
 - Un automóvil
 - Un alumno
 - Una cuenta bancaria
 - Un celular
 - Una mascota
- Un objeto tiene:
 - Estado
 - Comportamiento
 - Identidad

Componentes de la POO

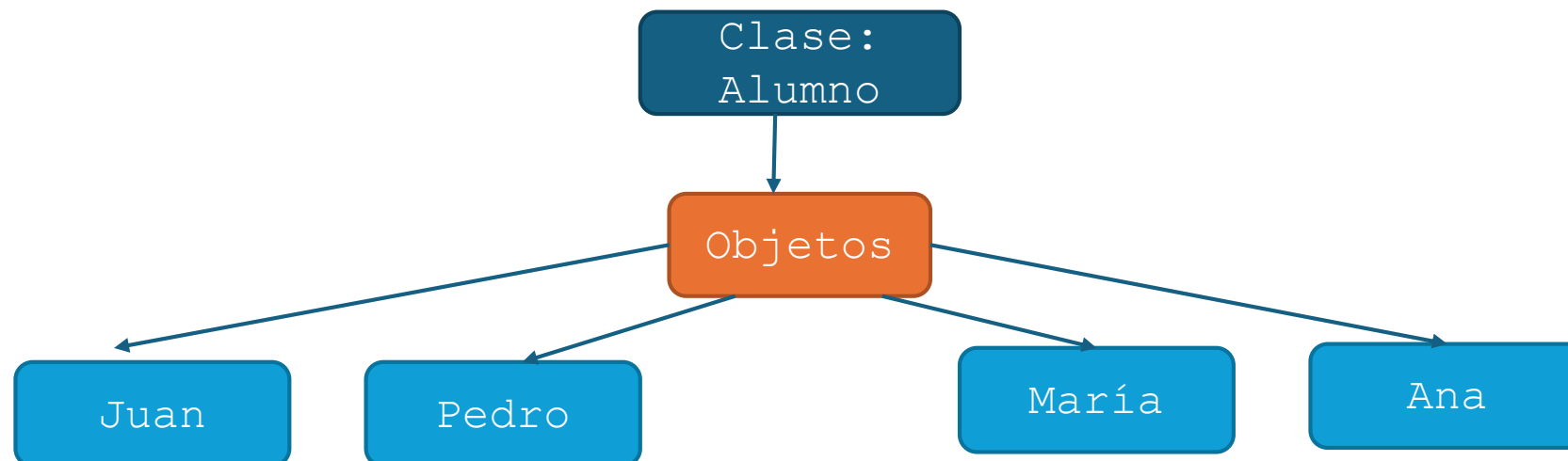
- Ejemplos cotidianos
- Una bicicleta
 - Estado
 - Color
 - Marca
 - Velocidad
 - Comportamiento
 - Frenar
 - Pedalear
 - Girar
- Una televisión
 - Estado
 - Canal
 - Volumen
 - Encendida
 - Comportamiento
 - Encender
 - Apagar
 - Cambiar canal

Componentes de la POO

- En Python
 - Creamos una clase.
 - Luego construimos objetos.
- `class Auto:`
- `pass`
- `auto1 = Auto()`
- `auto2 = Auto()`
- `auto1` y `auto2` son objetos distintos.

Componentes de la POO

- **¿Qué es una Clase?**
- Una clase es un molde.
 - Como un plano de una casa.
 - Con un plano podemos construir muchas casas.
- Con una clase podemos construir muchos objetos.



Componentes de la POO

- En Python:
- `class Alumno:`
- `pass`

- `alumno1 = Alumno()`
- `alumno2 = Alumno()`

Componentes de la POO

- **Atributos**

- Los atributos representan las características del objeto.

- Ejemplo

- Alumno

- Tiene

- Nombre

- Edad

- Carrera

- promedio

Componentes de la POO

- **Código**

- `class Alumno:`

- `def __init__(self,nombre,edad,carrera):`

- `self.nombre = nombre`

- `self.edad = edad`

- `self.carrera = carrera`

- **Crear objeto**

- `a1 = Alumno("Juan",20,"Informática")`

- **Acceder**

- `print(a1.nombre)`

- `print(a1.edad)`

- `print(a1.carrera)`

- **Resultado**

- Juan

- 20

- Informática

Componentes de la POO

- **Comportamiento**
 - Los comportamientos son las acciones del objeto.
- En programación son **métodos**.
- Ejemplo:
 - class Alumno:
 - def __init__(self,nombre):
 - self.nombre = nombre
 - def estudiar(self):
 - print(self.nombre,"está estudiando")
- **Uso**
 - a = Alumno("Carlos")
 - a.estudiar()
- **Resultado**
 - Carlos está estudiando

Componentes de la POO

- Otro ejemplo:
- class Perro:
 - def ladrar(self):
 - print("Guau")
- **Uso**
 - p = Perro()
 - p.ladrar()
- **Resultado**
 - Guau

Componentes de la POO

- **Estado de un objeto**
 - El estado corresponde a los valores actuales de sus atributos.
 - Ejemplo:
- **Auto**
 - Marca = Toyota
 - Color = Azul
 - Velocidad = 80
- **Si cambia la velocidad...**
 - Marca = Toyota
 - Color = Azul
 - Velocidad = 100
- **Cambió el estado.**

Componentes de la POO

- **Código**

- class Auto:
- def __init__(self):
 - self.velocidad = 0
- def acelerar(self):
 - self.velocidad += 20

- **Uso**

- auto = Auto()
- print(auto.velocidad)
- auto.acelerar()
- print(auto.velocidad)

- **Resultado**

- 0
- 20

Componentes de la POO

- **Accesibilidad**

- En Python existen tres niveles por convención:

- **Público**

- self.nombre
- Se puede acceder desde cualquier parte.

- **Protegido**

- self._nombre
- Por convención, indica que no debería accederse directamente desde fuera de la clase.

- **Privado**

- self.__saldo
- Python realiza *name mangling* para dificultar el acceso directo.

Componentes de la POO

- **Name mangling** es el proceso mediante el cual Python transforma automáticamente un atributo llamado **__atributo** en **_NombreDeLaClase__atributo**.
- Esto ayuda a evitar conflictos de nombres, especialmente cuando una clase es heredada, pero **no impide completamente el acceso al atributo**; simplemente lo hace menos directo.
- Es una herramienta para proteger la implementación interna de una clase, no un mecanismo de seguridad.

Componentes de la POO

- Ejemplo:
- class Cuenta:
- def __init__(self):
- self.__saldo = 1000
- cuenta = Cuenta()
- print(cuenta.__saldo)
- **Resultado**
 - Error

Componentes de la POO

- **La forma correcta es proporcionar métodos de acceso:**

- class Cuenta:

- def __init__(self):
 - self.__saldo = 1000
 - def consultar_saldo(self):
 - return self.__saldo
 - def depositar(self, monto):
 - if monto > 0:
 - self.__saldo += monto

- **Uso**

- cuenta = Cuenta()
 - print(cuenta.consultar_saldo())
 - cuenta.depositar(500)
 - print(cuenta.consultar_saldo())

- **Resultado**

- 1000
 - 1500

Propiedades de la Programación Orientada a Objetos

- Las cinco propiedades fundamentales son:
 - Abstracción
 - Encapsulamiento
 - Herencia
 - Polimorfismo
 - Sobrecarga

Propiedades de la Programación Orientada a Objetos

- **1. Abstracción**

- La abstracción consiste en mostrar únicamente lo importante y ocultar la complejidad interna.
- Ejemplo cotidiano:
- Cuando conduces un automóvil solo utilizas:
 - Volante
 - Pedales
 - Palanca de cambios
- No necesitas saber cómo funciona el motor internamente.

Propiedades de la Programación Orientada a Objetos

- En programación ocurre lo mismo.
- **class Cafetera:**
 - **def preparar_cafe(self):**
 - **print("Preparando café...")**
- El usuario solo llama:
- **cafetera = Cafetera()**
- **cafetera.preparar_cafe()**
- Resultado
 - **Preparando café...**
- No necesita conocer el proceso interno.

Propiedades de la Programación Orientada a Objetos

- **2. Encapsulamiento**

- Consiste en proteger los datos del objeto para evitar modificaciones indebidas.

- Ejemplo:

- class CuentaBancaria:

- def __init__(self):
 - self.__saldo = 0
 - def depositar(self, monto):
 - if monto > 0:
 - self.__saldo += monto
 - def retirar(self, monto):
 - if 0 < monto <= self.__saldo:
 - self.__saldo -= monto
 - def consultar_saldo(self):
 - return self.__saldo

- Uso:

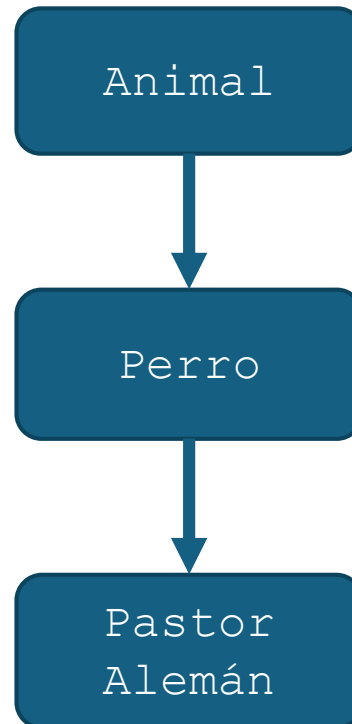
- cuenta = CuentaBancaria()
- cuenta.depositar(500)
- cuenta.retirar(100)
- print(cuenta.consultar_saldo())
- Resultado
 - 400

Propiedades de la Programación Orientada a Objetos

- **3. Herencia**

- Permite que una clase reutilice atributos y métodos de otra.

- Ejemplo:



Propiedades de la Programación Orientada a Objetos

- **Código:**
 - `class Animal:`
 - `def comer(self):`
 - `print("Estoy comiendo")`
 - `class Perro(Animal):`
 - `def ladrar(self):`
 - `print("Guau")`
- **Uso:**
 - `p = Perro()`
 - `p.comer()`
 - `p.ladrar()`
- **Resultado**
 - Estoy comiendo
 - Guau

Propiedades de la Programación Orientada a Objetos

- **4. Polimorfismo**

- El mismo método puede comportarse de manera diferente según el objeto.
- ```
class Perro:
```
- ```
    def hablar(self):
```
- ```
 print("Guau")
```
- ```
class Gato:
```
- ```
 def hablar(self):
```
- ```
        print("Miau")
```
- **Uso:**
- ```
animales = [Perro(), Gato()]
```
- ```
for animal in animales:
```
- ```
 animal.hablar()
```
- **Resultado:**
- Guau
- Miau
- **No importa el tipo del objeto; ambos responden al mensaje hablar().**

# Propiedades de la Programación Orientada a Objetos

- 5. Sobrecarga
  - En Python no existe la sobrecarga tradicional de métodos como en Java o C++, donde se pueden definir varios métodos con el mismo nombre y distinta firma.
- Si se declara dos veces un método, la última definición reemplaza a la anterior.
- Sin embargo, Python ofrece alternativas:
  - Parámetros con valores por defecto.
  - \*args y \*\*kwargs.
  - Sobrecarga de operadores mediante métodos especiales (`__add__`, `__eq__`, etc.).

# Propiedades de la Programación Orientada a Objetos

- **Ejemplo con parámetros por defecto:**

- class Calculadora:
  - def sumar(self, a, b=0, c=0):
  - return a + b + c

- calc = Calculadora()

- print(calc.sumar(5))
- print(calc.sumar(5, 3))
- print(calc.sumar(5, 3, 2))

- Resultado

- 5
- 8
- 10

# Propiedades de la Programación Orientada a Objetos

- **Ejemplo de sobrecarga de operadores:**

- `class Punto:`
- `def __init__(self, x, y):`
- `self.x = x`
- `self.y = y`
- `def __add__(self, otro):`
- `return Punto(self.x + otro.x, self.y + otro.y)`
- `def __str__(self):`
- `return f'({self.x}, {self.y})'`
- `p1 = Punto(2, 3)`
- `p2 = Punto(4, 5)`
- `print(p1 + p2)`
- Resultado
  - (6, 8)

# Ejemplo Integrador

- `class Persona:`
- `def __init__(self,nombre):`
- `self.nombre = nombre`
- `def hablar(self):`
- `print("Hola")`
- `class Profesor(Persona):`
- `def enseñar(self):`
- `print("Estoy enseñando Python")`
- `class Estudiante(Persona):`
- `def estudiar(self):`
- `print("Estoy estudiando")`

- **Uso**
- `profe = Profesor("Carlos")`
- `alumno = Estudiante("Ana")`
- `profe.hablar()`
- `profe.enseñar()`
- `alumno.hablar()`
- `alumno.estudiar()`
- **Resultado**
- `Hola`
- `Estoy enseñando Python`
- `Hola`
- `Estoy estudiando`

# Resumen Conceptual

| Concepto        | Descripción                                                                                | Ejemplo                               |
|-----------------|--------------------------------------------------------------------------------------------|---------------------------------------|
| Clase           | Molde para crear objetos                                                                   | <code>class Auto:</code>              |
| Objeto          | Instancia de una clase                                                                     | <code>auto = Auto()</code>            |
| Atributo        | Característica del objeto                                                                  | <code>color, marca</code>             |
| Método          | Acción del objeto                                                                          | <code>acelerar()</code>               |
| Estado          | Valores actuales de los atributos                                                          | <code>Velocidad = 80 km/h</code>      |
| Abstracción     | Mostrar lo esencial y ocultar detalles                                                     | <code>preparar_cafe()</code>          |
| Encapsulamiento | Proteger los datos internos                                                                | <code>__saldo</code>                  |
| Herencia        | Reutilizar código entre clases                                                             | <code>Perro(Animal)</code>            |
| Polimorfismo    | Un mismo método con distintos comportamientos                                              | <code>hablar()</code> en Perro y Gato |
| Sobrecarga      | En Python se logra mediante parámetros opcionales, <code>*args</code> y métodos especiales | <code>sumar(a, b=0)</code>            |

# Actividades Prácticas

- Crear una clase Libro con los atributos título, autor y precio, y un método mostrar\_información().
- Crear una clase Empleado con un atributo privado \_\_salario y métodos para consultar y actualizar el salario validando que no sea negativo.
- Diseñar una jerarquía de clases con Vehículo como clase base y Automóvil y Motocicleta como clases derivadas.
- Implementar las clases Rectángulo y Círculo, ambas con un método calcular\_area(), y recorrer una lista de figuras llamando al mismo método para observar el polimorfismo.
- Crear una clase Calculadora que permita sumar una cantidad variable de números utilizando \*args.

# Conclusión

- La Programación Orientada a Objetos organiza el software alrededor de **objetos** que combinan datos y comportamientos. Este paradigma mejora la reutilización del código, facilita el mantenimiento y permite modelar problemas del mundo real de forma natural. Los pilares de la POO —**abstracción, encapsulamiento, herencia y polimorfismo**—, junto con las técnicas que Python ofrece para simular la sobrecarga, constituyen la base para desarrollar aplicaciones escalables y robustas. Dominar estos conceptos permitirá afrontar con éxito las siguientes unidades del curso, donde se construirán sistemas más complejos aplicando estos principios.



